SQL command list

1. To create a database

```
CREATE DATABASE database_name;
```

Ex.
```
CREATE DATABASE student;
```

2. To delete a database

```
DROP DATABASE databasename;
```

Ex.
```
DROP DATABASE student;
```

3. To create a table under a database

```
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,
    column3 datatype,
   ....
);
```

Ex.
```
CREATE TABLE Persons (
    PersonID int,
    LastName varchar(255),
    FirstName varchar(255),
    Address varchar(255),
    City varchar(255)
);
```

```
Create table students (
studentId int,
name varchar(100),
department varchar(15),
emailid varchar(15),
city varchar(35)
);
```

create table student (student_id int(10), first_name varchar(30), last_name varchar(30), age int(10), email varchar(20));

4. To delete a table

```
DROP TABLE table_name;

Ex. DROP TABLE Persons;
```

5. To change or rename a table name

ALTER TABLE old_table_name RENAME TO new_table_name;

6. To add a new column in a table

```
ALTER TABLE table_name
ADD column_name datatype;

Ex. ALTER TABLE Customers
ADD Email varchar(255);
```

7. To delete a column in a table

```
ALTER TABLE table_name
DROP COLUMN column_name;

Ex. ALTER TABLE customers
DROP COLUMN email;
```

8. To change a column name

```
ALTER TABLE table_name
change column old_name new_name datatype(size);

Or

ALTER TABLE table_name
rename column old_name to new_name datatype(size);
```

9. To change a column type

```
ALTER TABLE table_name
modify column_name columntype(size);
```

10. Insert a record in a table

```
INSERT INTO table_name (column1, column2, column3, ...)
        VALUES (value1, value2, value3, ...);
```

Ex. `INSERT INTO student (studentId,fname,lname,age,email)
VALUES (190001,"Mohammad","Ibrahim",22,"ibrahim22@gmail.com");`

11. `Insert into students (studentId,name,department,emailid,city)
Values (2001, "Abir", "ICT", "abir@gmail.com", "Tangail");`

Insert a record in a table

```
INSERT INTO table_name (column1, column2, column3, ...)
        VALUES (value1, value2, value3, ...);
```

Ex. `INSERT INTO student (studentId,fname,lname,age,email)
VALUES (190001,"Mohammad","Ibrahim",22,"ibrahim22@gmail.com");`

12. Select all from a table

```
SELECT * FROM table_name;
```

Ex. `SELECT * FROM student;`

13. Select all from a table with limit

```
SELECT * FROM student limit 5;
```

14. Select last n rows from table

select * from table
order by column name desc limit rows;

select * from customer order by customerNo desc limit 3;

15. Select specific columns from a table

```
SELECT column1, column2, ...
FROM table_name;
```

```
Ex. SELECT studentId, fname, email
FROM student;
```

16. Select specific columns from a table with limit

```
SELECT column1, column2, ...
FROM table_name limit 5;
```

```
Ex. SELECT studentId, fname, email
FROM student limit 5;
```

17. SELECT DISTINCT column1, column2, ...
    FROM table_name;

    Where the coumn1 and coumn2 must have similar values at least twice

18. If a single column have similar values at least two times then

    Select distinct(column name) from table name;

19. Where condition in select

```
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Operator	Description
=	Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
<>	Not equal. **Note:** In some versions of SQL this operator may be written as !=
BETWEEN	Between a certain range
LIKE	Search for a pattern
IN	To specify multiple possible values for a column

i. Ex. select customerName,purItem,bill from customer where customerNo="cus-20004";
ii. select * from customer where bill>200;
iii. select * from customer where bill< 500;
iv. select * from customer where bill>=200;
v. select * from customer where bill<=500;
vi. select * from customer where bill !=500;
vii. select * from customer where bill between 200 and 500;
viii. select * from customer where customerName like "r%";
ix. select * from customer where customerName like "%r";
x. select * from customer where purItem in ("mafler","dettol","lungi");

20. AND, OR, NOT

AND Syntax

```
SELECT column1, column2, ...
FROM table_name
WHERE condition1 AND condition2 AND condition3 ...;
```

```
Ex. SELECT * FROM Customers
WHERE Country='Germany' AND City='Berlin';
```

OR Syntax

```
SELECT column1, column2, ...
FROM table_name
WHERE condition1 OR condition2 OR condition3 ...;

Ex. SELECT * FROM Customers
WHERE City='Berlin' OR City='München';
```

NOT Syntax

```
SELECT column1, column2, ...
FROM table_name
WHERE NOT condition;

Ex. SELECT * FROM Customers
WHERE NOT Country='Germany';
```

select * from customer where not customerName="obaidul kader";

21. Order by syntax

```
SELECT column1, column2, ...
FROM table_name
ORDER BY column1, column2, ... ASC|DESC;
```

select * from customer order by customerName;

select * from customer order by bill desc;

22. Null values

```
SELECT column_names
FROM table_name
WHERE column_name IS NULL;
```

select * from customer where contactAddress is null;
select * from customer where contactAddress is not null;

update customer set branch=null where customerNo="cus-20004";

select customerNo,customerName from customer where contactAddress is not null;

select customerNo,customerName from customer where contactAddress is null;

23. Update commands

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

```
UPDATE Customers
SET ContactName = 'Alfred Schmidt', City= 'Frankfurt'
WHERE CustomerID = 1;
```

```
UPDATE Customers
SET ContactName='Juan'
WHERE Country='Mexico';
```

```
Update customer
Set shopName="Twelve,Tangail";
```

update customer set contactAddress=Null where customerName="rabir";

24. Delete commands

Delete all

```
DELETE FROM table_name;
```

```
DELETE FROM Customers; or
```

```
TRUNCATE TABLE table_name;
```

```
DELETE FROM table_name WHERE condition;
```

```
DELETE FROM Customers WHERE CustomerName='Alfreds Futterkiste';
```

25. Limit

Select * from table_name limit 3; (ascending order)

Select * from table_name order by column_name desc limit 3; (decending order)

Select * from table name limit 3 offset 2;

or

select * from abir limit 2,3;

26. Mysql functions

i. Min() and Max() function

select min(bill) from customer; (without column renaming)

select min(bill) as “minimum bill” from customer; (with column renaming)

select min(bill) as minimum_bill from customer; (with column renaming)

select max(bill) as maximum_bill from customer;

```
SELECT MIN(column_name)
FROM table_name
WHERE condition;
```

ii. COUNT() Function

Find the total number of rows in the table

select count(*) from customer;

select count(*) "Total number of records" from customer;

Count don’t count NULL values

select count(contactAddress) from customer;

Specific column

select count(customerNo) from customer;

select count(bill) from customer;

Count with where condition

select count(bill) from customer where bill>=100;

Count with Ignore duplicates

select count(distinct bill) from customer;

Count Use an Alias

select count(distinct bill) as “distinct bill” from customer;

Group By with Count

select count(distinct bill),contactAddress from customer group by contactAddress;

select count(distinct bill),customerName from customer group by customerName;

select count(distinct bill),shopName from customer group by shopName;

iii. SQL SUM() Function

select sum(bill) from customer;

select sum(bill) from customer where customerNo="cus-20003";

apply 10% vat-tax of the customer bill with sum function as

select sum(bill+(bill*10)/100) from customer where customerNo="cus-20003";

SUM() with GROUP BY

select customerNo,sum(bill) from customer group by customerNo;

iv. **SQL AVG() Function**

select avg(bill) from customer;

select avg(bill) from customer where customerNo="cus-20003";

Higher Than Average/Nested Quesry

select * from customer where bill> (select avg(bill) from customer);

AVG() with GROUP BY

select customerNo, avg(bill) as "Average bill" from customer group by customerNo;

27. Wildcard characters

select * from customer where customerName like “_____r”;

select * from customer where customerName like "r%r";

28. Exists

The EXISTS operator is used to test for the existence of any record in a subquery.

The EXISTS operator returns TRUE if the subquery returns one or more records.

```
SELECT column_name(s)
FROM table_name
WHERE EXISTS
(SELECT column_name FROM table_name WHERE condition);
```

select customerNo,purItem from customer where exists (select bill from customer where bill>1000);

select * from customer where exists (select purItem from customer where purItem in("puff rice","medel"));

29. Copy a Table all contents and create a new named table

create table customerBackup select * from customer;

from another database to a new database with selective column

create table customerBackup select customerNo,customerName,bill from edgeict.customer;

create a table with selective column and renaming column and without record insertion
create table customerr as select customerNo as cus_id, customerName as cus_name, bill from customer where 1=2;

30. Insert from another table

insert into customerBackup select customerNo,customerName,bill from edgeict.customer;

insert into customerBackup select customerNo,customerName,bill from edgeict.customer limit 3;

31. Case in mysql

```
CASE
    WHEN condition1 THEN result1
    WHEN condition2 THEN result2
    WHEN conditionN THEN resultN
    ELSE result
END;
```

```
SELECT OrderID, Quantity,
CASE
    WHEN Quantity > 30 THEN 'The quantity is greater than 30'
    WHEN Quantity = 30 THEN 'The quantity is 30'
    ELSE 'The quantity is under 30'
END AS QuantityText
FROM OrderDetails;
```

```
select customerNo,customerName,bill,
   -> case
   -> when bill>100 then "Middle economic person"
   -> when bill<100 then "Poor"
   -> else "Between middle and poor"
   -> end as billtext
   -> from customer;
```

32. IfNull function

The MySQL IFNULL() function lets you return an alternative value if an expression is NULL:

```
SELECT ProductName, UnitPrice * (UnitsInStock +
IFNULL(UnitsOnOrder, 0))
FROM Products;
```

select customerNo, date_and_time, ifnull(price,0) from supplier;

select customerNo,customerName,ifnull(bill,0),ifnull(contactAddress,"ICT, MBSTU") from customerinfo;

33. SQL ANY Operator

The ANY operator:

- returns a boolean value as a result
- returns TRUE if ANY of the subquery values meet the condition

ANY means that the condition will be true if the operation is true for any of the values in the range.

```
select customerName,purItem from customer where bill = any (
select bill from customer where bill=200);

select customerName,purItem from customer where bill = all (
select bill from customer where bill=200);
```

34. Primary key foreign key and null constraints

```
create table supplier (customerNo varchar(40) primary key,
                       supplierNo varchar(40) not null,
                       Date_and_Time DATETIME,
                       price int);

create table billing ( customerNo varchar(40),
                       date DATE,
                       bill int,
    foreign key(customerNo) references supplier(customerNo));
```

the foreign key table won”t accept any customerNo that would not present in supplier table customerNo.

35. Check constraint

The CHECK constraint is used to limit the value range that can be placed in a column.

```
CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    CHECK (Age>=18)
);

alter table customerinfoB modify bill int check (bill>0);
```

36. Default

```
CREATE TABLE Orders (
ID int NOT NULL,
OrderNumber int NOT NULL,
OrderDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
```

37. Auto increment

MySQL uses the AUTO_INCREMENT keyword to perform an auto-increment feature.

```
CREATE TABLE Persons (
Personid int NOT NULL AUTO_INCREMENT,
Name varchar(255) NOT NULL,
Age int,
PRIMARY KEY (Personid)
);
```

Create table autupdate (
Id int Auto_increment primary key,
Data varchar(255),
Last_modified timestamp default current_timestamp on update current_timestamp);

38. Cartesian product of two table

select customerNo, purItem, id,age from customer,stu;

39. UNION Operator

```
SELECT column_name(s) FROM table1
UNION
SELECT column_name(s) FROM table2;
```

select customerNo,purItem from customer union all select id,age from stu;

```
mysql> select * from customer;
+------------+--------------+-----------+------+-----------------+----------------+
| customerNo | customerName | purItem   | bill | contactAddress  | branch         |
+------------+--------------+-----------+------+-----------------+----------------+
| cus-20002  | kabir        | lungi     |  500 | NULL            | swopno,tangail |
| cus-20003  | akbar        | towel     |  200 | dhanmodi,dhaka  | swopno,tangail |
| cus-20004  | Rahman       | noodles   |  200 | dhanmodi,dhaka  | swopno,tangail |
| cus-20004  | Rahman       | puff rice |  100 | dhanmodi,dhaka  | swopno,tangail |
| cus-20003  | akbar        | dettol    |   60 | dhanmodi,dhaka  | swopno,tangail |
| cus-20005  | rabir        | chips     |   50 | NULL            | swopno,tangail |
| cus-20001  | obaidul kader | mafler   | 1000 | dhanmondi,dhaka | swopno,tangail |
+------------+--------------+-----------+------+-----------------+----------------+
7 rows in set (0.00 sec)

mysql> select * from supplier;
+------------+------------+---------------------+-------+
| customerNo | supplierNo | Date_and_Time       | price |
+------------+------------+---------------------+-------+
| cus-20001  | sup-20001  | 2024-09-12 11:12:45 |   545 |
| cus-20002  | sup-20002  | 2024-09-12 12:01:00 |   245 |
| cus-20003  | sup-20003  | 2024-09-12 12:25:00 |   845 |
| cus-20004  | sup-20001  | 2024-09-12 04:50:44 |  NULL |
+------------+------------+---------------------+-------+
```

40. Natural join

select customerNo, purItem,id,age from customer,stu where customer.customerName=stu.name;

select customerNo,supplierNo, customerName,price from customer natural join supplier;

41. Inner Join without condition

select customerName,purItem,supplierNo,price from customer inner join supplier;

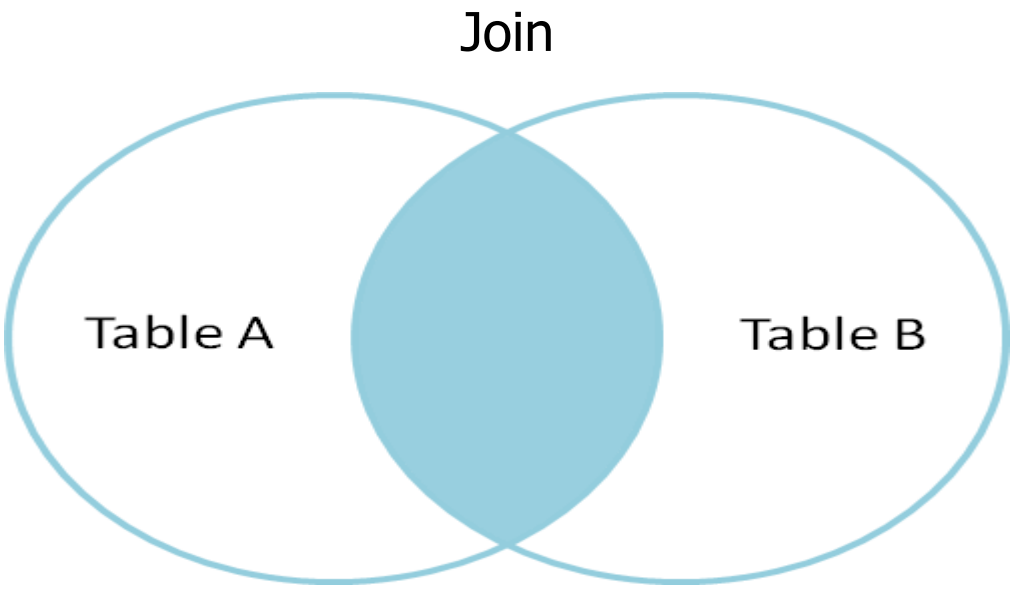
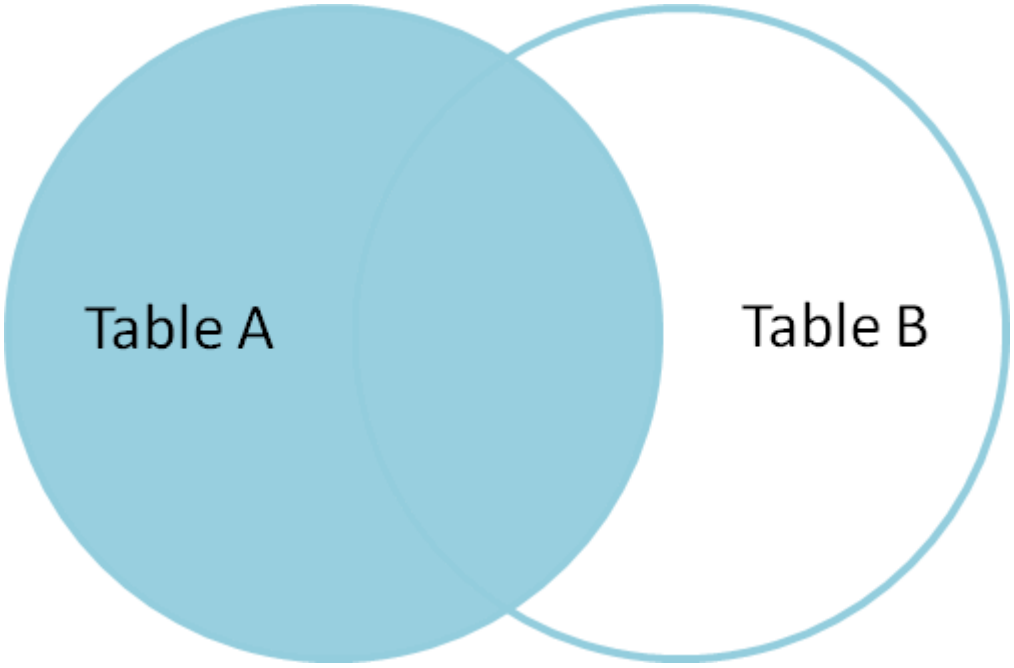


Fig. Inner Join

select customerName,purItem,supplierNo,price from customer inner join supplier on customer.customerNo=supplier.customerNo;

42. Left Join/Left Outer Join

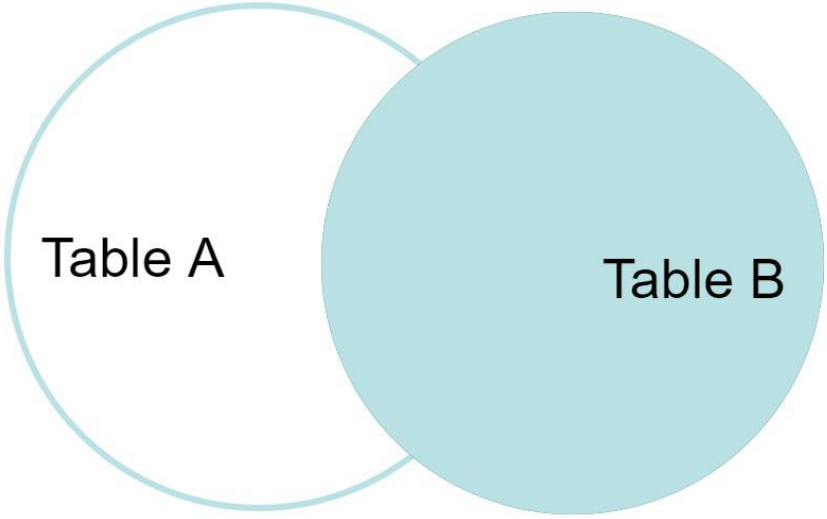
It selects the value that common to the left table or first mentioned table.



select customerName,purItem,supplierNo,price from customer left join supplier on customer.customerNo=supplier.customerNo;

select supplierNo,price,customerName,purItem from supplier left join customer on supplier.customerNo=customer.customerNo;

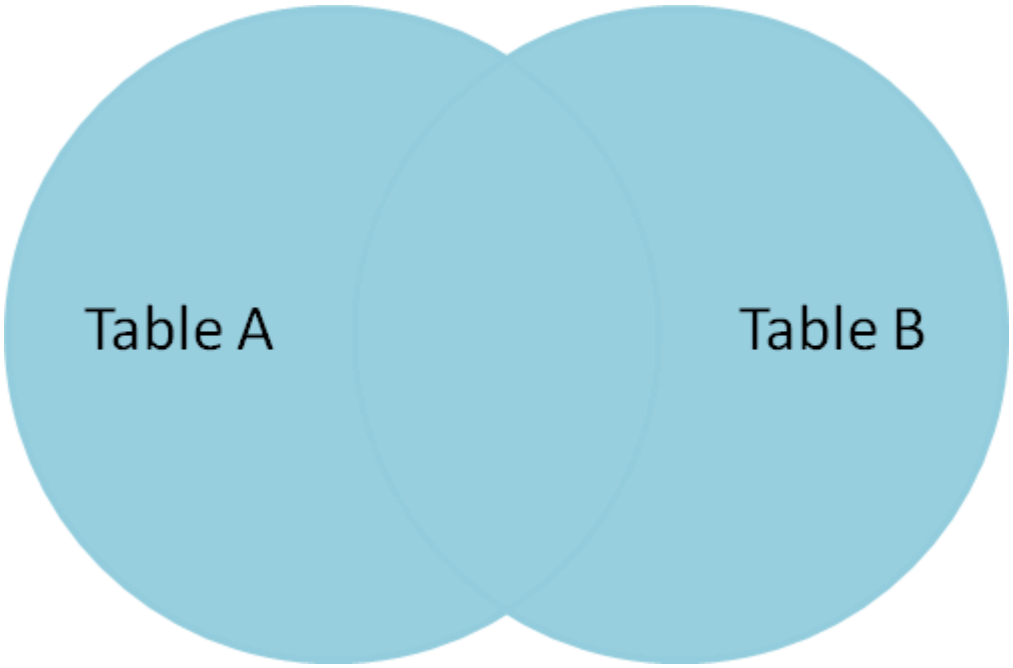
43. Right join/Right Outer join



select customerName,purItem,supplierNo,price from customer right join supplier on customer.customerNo=supplier.customerNo;

select supplierNo,price,customerName,purItem from supplier right join customer on supplier.customerNo=customer.customerNo;

44. Full join



1.
select customerName,purItem,supplierNo,price from customer left join supplier on customer.customerNo=supplier.customerNo
union
select customerName,purItem,supplierNo,price from customer right join supplier on customer.customerNo=supplier.customerNo;

2.

select customerName,purItem,supplierNo,price from customer right join supplier on customer.customerNo=supplier.customerNo
union
select customerName,purItem,supplierNo,price from customer left join supplier on customer.customerNo=supplier.customerNo;

3. select * from customer left join supplier on customer.customerNo=supplier.customerNo union select * from customer right join supplier on customer.customerNo=supplier.customerNo;

45. Cross join

select * from customer cross join supplier;

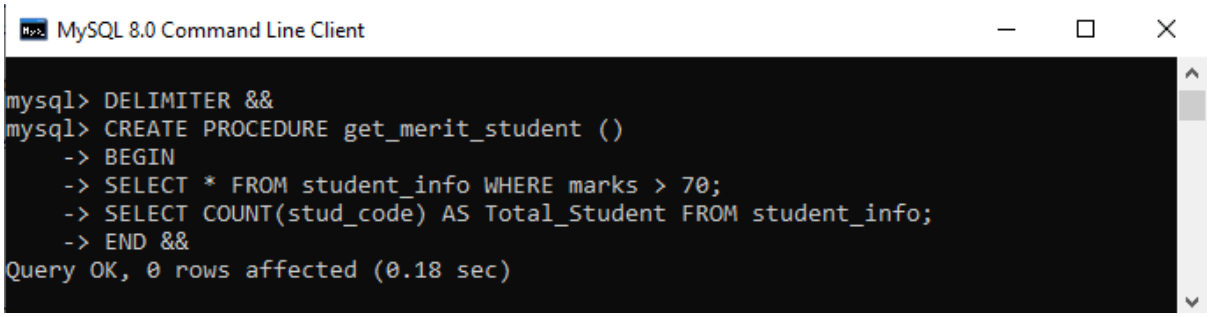
select customerName,purItem,supplierNo,price from customer cross join supplier on customer.customerNo=supplier.customerNo;

46. Create procedure

A stored procedure is a prepared SQL code that you can save, so the code can be reused over and over again.

List out all procedure in a database
show procedure status where db="edgeict";

To delete a procedure
Drop procedure procedure_name;



i. delimiter &&
mysql> create procedure deleteAll()
-> begin
-> delete from edgeict;
-> end &&

Call/execute a procedure

call deleteAll();

i. delimiter &&
mysql> create procedure bill_count()
-> begin
-> select count(bill) from customer where bill>=500;
-> end &&

ii. delimiter &&
mysql> create procedure customerInfo(IN cname varchar(50))
-> begin
-> select * from edgeict.customer where customer.customerName=cname;
-> end&&

iii. delimiter &&
mysql> create procedure totalBill `(IN inCustomerNo VARCHAR(15), OUT outBill FLOAT)
-> begin

-> SELECT SUM(bill) INTO outBill FROM customer
WHERE customerNo = inCustomerNo;
-> end &&

Call totalBill("cus-20003",@bill);
CALL totalBill(@customerNo, @total);
SELECT @total; -- to see the output

47. Trigger

https://www.javatpoint.com/mysql-trigger

A trigger in MySQL is a set of SQL statements that reside in a system catalog. It is a special type of stored procedure that is invoked automatically in response to an event. Each trigger is associated with a table, which is activated on any DML statement such as INSERT, UPDATE, or DELETE.

Generally, triggers are of two types according to the SQL standard: **row-level triggers** and **statement-level triggers**.

Row-Level Trigger: It is a trigger, which is activated **for each row by a triggering statement** such as insert, update, or delete. For example, if a table has inserted, updated, or deleted multiple rows, the row trigger is fired automatically for each row affected by the insert, update, or delete statement.

Statement-Level Trigger: It is a trigger, which is fired once for each event that occurs on a table regardless of how many rows are inserted, updated, or deleted.

Types of Triggers in MySQL?
We can define the maximum **six types** of actions or events in the form of triggers:

1. **Before Insert:** It is activated before the insertion of data into the table.
2. **After Insert:** It is activated after the insertion of data into the table.
3. **Before Update:** It is activated before the update of data in the table.
4. **After Update:** It is activated after the update of the data in the table.
5. **Before Delete:** It is activated before the data is removed from the table.
6. **After Delete:** It is activated after the deletion of data from the table.

```
CREATE TRIGGER trigger_name  trigger_time trigger_event
ON table_name FOR EACH ROW
BEGIN
   --variable declarations
   --trigger code
END;
```

Ex.

```
1. DELIMITER &&
2. mysql> Create Trigger customerStatus
3. BEFORE INSERT ON customer FOR EACH ROW
4. BEGIN
5. IF NEW.bill >= 500 THEN SET NEW.customer_status = “Rich”;
6. END IF;
7. END //
```

Check trigger status for a table

show triggers like "customer";

delete a trigger

drop trigger edgeict.customerStatus;

I. Before Insert Trigger

delimiter &&
mysql> create trigger customerTag
-> before insert on customer for each row
-> begin
-> if new.bill>=500 then set new.customer_status="Rich";
-> end if;
-> end&&

insert into customer (customerNo,customerName,bill) values ("cus-20007","lily",600);

delimiter &&
mysql> create trigger customerTag
-> before insert on customer for each row
-> begin
-> if new.bill>=500 then set new.customer_status="Rich";
->elseif (new.bill>=200 and new.bill<500) then set new.customer_status="Middle_class";
-> else set new.customer_status="Poor";
-> end if;
-> end&&

```
CREATE TRIGGER upd_check BEFORE UPDATE ON account
       FOR EACH ROW
       BEGIN
           IF NEW.amount < 0 THEN
               SET NEW.amount = 0;
           ELSEIF NEW.amount > 100 THEN
               SET NEW.amount = 100;
           END IF;
              END;//
```

II. After Insert trigger

```
CREATE TRIGGER trigger_name
AFTER INSERT
ON table_name FOR EACH ROW
trigger_body ;
```

```
-- Create the employees table
CREATE TABLE employees (
    employee_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    hire_date DATE
);

-- Create the employee_audit table
CREATE TABLE employee_audit (
    audit_id INT PRIMARY KEY AUTO_INCREMENT,
    employee_id INT,
    action VARCHAR(50),
    action_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

-- **Create an AFTER INSERT trigger**

```
DELIMITER $$

CREATE TRIGGER after_employee_insert
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
    INSERT INTO employee_audit (employee_id, action)
    VALUES (NEW.employee_id, 'INSERT');
END$$
DELIMITER ;
INSERT INTO employees (employee_id, first_name, last_name, hire_date) VALUES (1, 'John',
'Doe', '2023-10-01');

 -- Check the employee_audit table
SELECT * FROM employee_audit;
```

III. Before Update trigger

```
-- Create the products table
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100),
    price DECIMAL(10, 2)
);

-- Create the price_change_log table
CREATE TABLE price_change_log (
    log_id INT PRIMARY KEY AUTO_INCREMENT,
    product_id INT,
    old_price DECIMAL(10, 2),
    new_price DECIMAL(10, 2),
    change_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
-- Create a BEFORE UPDATE trigger
DELIMITER $$

CREATE TRIGGER before_price_update
BEFORE UPDATE ON products
FOR EACH ROW
BEGIN
    -- Ensure the new price is not negative
    IF NEW.price < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Price cannot be negative';
    END IF;

    -- Log the price change
    IF OLD.price != NEW.price THEN
        INSERT INTO price_change_log (product_id, old_price, new_price)
        VALUES (OLD.product_id, OLD.price, NEW.price);
    END IF;
END$$

DELIMITER ;
```

```
  -- Insert a product
INSERT INTO products (product_id, product_name, price)
VALUES (1, 'Laptop', 1200.00);

-- Update the product's price
UPDATE products
SET price = 1300.00
WHERE product_id = 1;

-- Check the price_change_log table
SELECT * FROM price_change_log;

-- Try to set a negative price (this will fail)
UPDATE products
SET price = -200.00
WHERE product_id = 1;
```

iv. After update trigger

```
CREATE TABLE employees (
   employee_id INT PRIMARY KEY,
   first_name VARCHAR(50),
   last_name VARCHAR(50),
   salary DECIMAL(10, 2)
);


CREATE TABLE employees_audit (
   audit_id INT AUTO_INCREMENT PRIMARY KEY,
   employee_id INT,
   old_salary DECIMAL(10, 2),
   new_salary DECIMAL(10, 2),
   updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);


DELIMITER $$

CREATE TRIGGER after_employee_update
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
   -- Check if the salary has changed
   IF OLD.salary != NEW.salary THEN
      INSERT INTO employee_audit (employee_id, old_salary, new_salary)
      VALUES (OLD.employee_id, OLD.salary, NEW.salary);
   END IF;
END$$

DELIMITER ;


Insert few data on employees table

Update employees table
update employees set salary=45000.00 where employee_id=1;

check employees table
select * from employees;

check employee_audit table
select * from employee_audit;
```

v. Before delete Trigger

```
   CREATE TABLE employees (
   employee_id INT PRIMARY KEY,
   first_name VARCHAR(50),
   last_name VARCHAR(50),
   department VARCHAR(50)
);

   CREATE TABLE deleted_employees_log (
   log_id INT AUTO_INCREMENT PRIMARY KEY,
   employee_id INT,
   deleted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DELIMITER $$

CREATE TRIGGER before_employee_delete
BEFORE DELETE
ON employees
FOR EACH ROW
BEGIN
   INSERT INTO deleted_employees_log (employee_id)
   VALUES (OLD.employee_id);
END;
```

Insert few data on employees table

delete from employees where employee_id=3;

check data on deleted_employees_log

VI. **After delete Trigger**

```
CREATE TABLE employees (
   employee_id INT PRIMARY KEY,
   employee_name VARCHAR(100),
   department VARCHAR(100)
);

CREATE TABLE employee_audit (
   audit_id INT PRIMARY KEY AUTO_INCREMENT,
   employee_id INT,
   employee_name VARCHAR(100),
   department VARCHAR(100),
   deleted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);


CREATE TRIGGER after_employee_delete
AFTER DELETE
ON employees
FOR EACH ROW
BEGIN
   INSERT INTO employee_audit (employee_id, employee_name, department)
   VALUES (OLD.employee_id, OLD.employee_name, OLD.department);
END;
```

MySQL workbench

Video Tutorial from youtube

1. **Create database and tables**
   https://www.youtube.com/watch?v=wALCw0F8e9M&list=PLSwH4ViBDl2RRKkR2608p4cYa4321euG5&index=1

ORACLE installation

1. **Link**
   https://www.oracle.com/database/technologies/appdev/xe.html

   https://www.youtube.com/watch?v=iv7qlWMsHFc

   https://www.tutorialspoint.com/plsql/plsql_environment_setup.htm

   https://www.tutorialspoint.com/plsql/plsql_environment_setup.htm

   oracle login as

   /as sysdba

   https://oracledbwr.com/creating-oracle-21c-database-using-dbca-method/#:~:text=Step%3A%2D%201%20Select%20Create,choose%20the%20database%20deployment%20type.

connect XE as sysdba;

User creation

CREATE USER username IDENTIFIED BY password;

CREATE USER C##abir IDENTIFIED BY edgeict;
GRANT CONNECT, RESOURCE TO C##abir;

User login

Connect C##abir
Password: edgeict

SHOW USER;

Grant a sysdba access to a user

alter session set container = XEPDB1;

GRANT SYSDBA TO C##abir;

Check for SYSDBA grant access

SELECT * FROM V$PWFILE_USERS WHERE USERNAME = 'C##ABIR';

To show the current database (as sysdba login)
SELECT name FROM v$database;

Show connected databases (plugable database)

show pdbs

connect XEPDB1 database command

connect XEPDB1 as sysdba

Current version checking
select * from v$version;
Show all tables under XEPDB1

```
SELECT table_name
FROM all_tables;
```

Create a table under this database

```
create table student(
id int,
name varchar(30),
dept varchar(10));
```

describe student;

insert a record into the file

insert into student values (101,'Rohan','ICT');

SQL> insert into student values (101,'Rohan','ICT');
insert into student values (101,'Rohan','ICT')
*****
ERROR at line 1:
ORA-01950: no privileges on tablespace 'USERS'

Solution[in sysdba mode]:
ALTER USER C##rabir QUOTA UNLIMITED ON users;

insert multiple records at a time

1. **Insert into**

INSERT INTO table_name (column1, column2, column3) VALUES (value1a, value2a, value3a);

INSERT INTO table_name (column1, column2, column3) VALUES (value1b, value2b, value3b);
INSERT INTO table_name (column1, column2, column3) VALUES (value1c, value2c, value3c);

insert into student (id,name,dept) values (103,’rajib’,’econ’);
insert into student (id,name,dept) values (104,’abrar’,’tex’);

2. Insert all

```
INSERT ALL
INTO table_name (column1, column2, column3) VALUES (value1a, value2a, value3a)
INTO table_name (column1, column2, column3) VALUES (value1b, value2b, value3b)
INTO table_name (column1, column2, column3) VALUES (value1c, value2c, value3c)
SELECT * FROM dual;
```

```
insert all
into student (id,name,dept) values (107,’akhi’,’BGE’)
into student (id,name,dept) values (108, ’sumona’,’CPS’)
into student (id,name,dept) values (109, ’angona’,’phar’)
select * from dual;
```

Insert using pl/sql block

Open a file using sql command line and paste the following section

```
BEGIN
insert into student (id,name,dept) values (110,'akij','phy');
insert into student (id,name,dept) values (111,'rika','chem');
insert into student (id,name,dept) values (112,'rita','stat');
END;
/
```

And execute it

Rollback

rollback;

Rollback complete.[undo the last insert operation]

Commit

commit;

Commit complete.[permanently saves the changes]

Create a table from a table

I. Creating a Table from a table with Data

To copy both the structure and the data from the existing table:

```
CREATE TABLE new_table_name AS
SELECT * FROM existing_table_name;

Create table studentbackup as select * from student;
```

II. Creating a Table from a table with only column and column rename

create table studentinfo (student_id,stu_name,department) as select id,name,dept from student where 1=2;

Insert data into a table from another table

insert into studentinfo select id,name,dept from student where name like 'r%';

Delete command

Delete All

DELETE FROM student;

Delete with condition
DELETE FROM student WHERE student_id = 101;

DELETE FROM student WHERE department = 'ICT' AND name = 'Rohan';

How to execute a block in pl/sql
https://www.youtube.com/watch?v=cEmByITgCLs

Using PL/SQL block

To create the procedure in sql file
Sql> ed F:\Abir_MBSTU\EDGE_project\Lab\pl_sql\delete.sql

Paste the below code

```
BEGIN
  DELETE FROM student
  WHERE id = 102;

  IF SQL%ROWCOUNT > 0 THEN
    DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' row(s) deleted.');
  ELSE
    DBMS_OUTPUT.PUT_LINE('No rows deleted.');
  END IF;

  COMMIT;
END;
/
```

To run the sql

@F:\Abir_MBSTU\EDGE_project\Lab\pl_sql\delete.sql

Sql> set serveroutput on;

Again run

@F:\Abir_MBSTU\EDGE_project\Lab\pl_sql\delete.sql

Update

update table_name

Order By

select * from student order by ID; [Ascending order]

select * from student order by ID desc; [Descending order]

Subtype

subtype is a subset of another data type, which is called its base type.

```
DECLARE
   SUBTYPE name IS char(20);
   SUBTYPE message IS varchar2(100);
   salutation name;
   greetings message;
BEGIN
```

```
   salutation := 'Reader ';
   greetings := 'Welcome to the World of PL/SQL';
   dbms_output.put_line('Hello ' || salutation || greetings);
END;
/
```

Variable Declaration and usage

```
DECLARE
  a integer := 10;
  b integer := 25;
  c integer;
  f real;
BEGIN
  c := a + b;
  dbms_output.put_line('Value of a: ' || a);
  dbms_output.put_line('Value of b: ' || b);
  dbms_output.put_line('The sum of a and B is: ' || c);
  f := b/a;
  dbms_output.put_line('Value of b divides a is : ' || f);
END;
/
```

Constant declaration

```
DECLARE
  PI CONSTANT NUMBER := 3.141592654;
  radius number(5,2);
  dia number(5,2);
  circumference number(7, 2);
  area number (10, 2);
BEGIN
  radius := 9.5;
  dia := radius * 2;
  circumference := 2.0 * PI * radius;
  area := PI * radius * radius;

  dbms_output.put_line('Radius: ' || radius);
  dbms_output.put_line('Diameter: ' || dia);
  dbms_output.put_line('Circumference: ' || circumference);
  dbms_output.put_line('Area: ' || area);
END;
/
```

Loop example

```
DECLARE
  i number(1);
  j number(1);
BEGIN

  FOR i IN 1..3 LOOP

    FOR j IN 1..3 LOOP
      dbms_output.put_line('i is: '|| i || ' and j is: ' || j);
    END loop;
  END loop;
END;
/
```

String Example

```
DECLARE
  greetings varchar2(11) := 'hello world';
BEGIN
  dbms_output.put_line(UPPER(greetings));

  dbms_output.put_line(LOWER(greetings));

  dbms_output.put_line(INITCAP(greetings));

  /* retrieve the first character in the string */
  dbms_output.put_line ( SUBSTR (greetings, 1, 1));

  /* retrieve the last character in the string */
  dbms_output.put_line ( SUBSTR (greetings, -1, 1));

  /* retrieve five characters,
    starting from the seventh position. */
  dbms_output.put_line ( SUBSTR (greetings, 7, 5));

  /* retrieve the remainder of the string,
    starting from the second position. */
  dbms_output.put_line ( SUBSTR (greetings, 2));

  /* find the location of the first "e" */
  dbms_output.put_line ( INSTR (greetings, 'l'));
```

END;
/

Procedure

```
CREATE [OR REPLACE] PROCEDURE procedure_name
[(parameter_name [IN | OUT | IN OUT] type [, ...])]
{IS | AS}
BEGIN
  < procedure_body >
END procedure_name;
```

Show list of procedure

SELECT object_name
FROM user_objects
WHERE object_type = 'PROCEDURE';

Show internal code of a procedure

SELECT text
FROM all_source
WHERE name = 'GREETINGS'
AND type = 'PROCEDURE'
ORDER BY line;

Delete procedure

DROP PROCEDURE procedure_name;

1. **Simple procedure**

I.

CREATE OR REPLACE PROCEDURE greetings
AS
BEGIN
dbms_output.put_line('Hello World!');
END;
/

exec greetings;

II.

```
CREATE OR REPLACE PROCEDURE greet_employee (
   employee_name IN VARCHAR2
) AS
BEGIN
   DBMS_OUTPUT.PUT_LINE('Hello, ' || employee_name || '! Welcome
to the company.');
END;
/

exec greet_employee ('Abir');
```

III.

Create a procedure in pl/sql

```
    create table student(
    id int,
   name varchar(30),
   dept varchar(10));


create or replace procedure add_student (
  2  p_id IN INT,
  3  p_name IN VARCHAR,
  4  p_dept IN VARCHAR ) AS
  5  begin
  6  insert into student (ID, NAME, DEPT )
  7  values (p_id, p_name, p_dept);
  8  commit;
  9  end add_student;
 10  /


exec add_student ( 102, 'rajon', 'Mech' );

select * from student;
```

IN and OUT example

```
DECLARE
   a number;
   b number;
   c number;
```

```
PROCEDURE findMin(x IN number, y IN number, z OUT number) IS
BEGIN
   IF x < y THEN
      z:= x;
   ELSE
      z:= y;
   END IF;
END;
BEGIN
   a:= 23;
   b:= 45;
   findMin(a, b, c);
   dbms_output.put_line(' Minimum of (23, 45) : ' || c);
END;
/
```

In and OUT at the same time

```
DECLARE
   a number;
PROCEDURE squareNum(x IN OUT number) IS
BEGIN
  x := x * x;
END;
BEGIN
   a:= 23;
   squareNum(a);
   dbms_output.put_line(' Square of (23): ' || a);
END;
/
```

Function

```
CREATE OR REPLACE FUNCTION totalStudents
RETURN number IS
   total number(2) := 0;
BEGIN
   SELECT count(*) into total
   FROM student;

   RETURN total;
END;
/
```

To call the function

declare
  2 c number(2);

```
  3  begin
  4  c:=totalStudents();
  5  dbms_output.put_line('Total no. of student: ' || c);
  6  END;
  7  /
```

Trigger

SELECT TRIGGER_NAME, TABLE_NAME, STATUS FROM USER_TRIGGERS;

Create a table

```
create table customer (
  2  ID number(6),
  3  salary decimal(6,2));
```

```
CREATE OR REPLACE TRIGGER display_salary_changes
BEFORE DELETE OR INSERT OR UPDATE ON customer
FOR EACH ROW
WHEN (NEW.ID > 0)
DECLARE
   sal_diff number;
BEGIN
   sal_diff := :NEW.salary  - :OLD.salary;
   dbms_output.put_line('Old salary: ' || :OLD.salary);
   dbms_output.put_line('New salary: ' || :NEW.salary);
   dbms_output.put_line('Salary difference: ' || sal_diff);
END;
/
```

Insert into customer values (101,30000.00);

Update and check